

Chapter 7: Forms

Switch over to class-based views. Less code, less error prone.

Code

```
# blog/views.py
from django.views.generic import ListView, DetailView
from .models import Post

class BlogListView(ListView):
    model = Post
    template_name = "home.html"

class BlogDetailView(DetailView):
    model = Post
    template_name = "post_detail.html"
```

At the top, we import the generic views **ListView** and **DetailView**, along with our model, **Post**. We used **ListView** previously in the Message Board app, but **DetailView** is new. It is used for detail pages and has a format extremely similar to **ListView**. **We pass in the generic class-based view and define the model and template for both views.**

Code

```
# blog/urls.py
from django.urls import path
from .views import BlogListView, BlogDetailView # new

urlpatterns = [
    path("post/<int:pk>/", BlogDetailView.as_view(), name="post_detail"), # new
    path("", BlogListView.as_view(), name="home"), # new
]
```

Then, we update the second argument in both routes, specifying the view name and adding **as_view()** to transform the class into a callable view function.

Code

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block content %}
{% for post in post_list %}  <!-- new -->
<div class="post-entry">
  <h2><a href="{{ post.get_absolute_url }}">{{ post.title }}</a></h2>
  <p>{{ post.body }}</p>
</div>
{% endfor %}
{% endblock content %}
```

The final update is to our template. Previously, in our function-based list and detail views, we named the context object **posts** and **post** respectively. But for the GCBV ListView, you need to know that the default naming pattern is **<model>_list**; since our model is Post, the context object will be **post_list**.

Code

```
# blog/views.py
...
class BlogListView(ListView):
    model = Post
    template_name = "home.html"
    context_object_name = 'posts' # Change the context object name to 'posts'
...
```

Could rename context object if you want to.

The context object name in a `DetailView` also has a default value of either the model name (so `post` in this case) or `object`. That means we could leave our current template as is.

Code

```
<!-- templates/post_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="post-entry">
  <h2>{{ post.title }}</h2>
  <p>{{ post.body }}</p>
</div>
{% endblock content %}
```

Or we could change `post` to `object` like this:

Code

```
<!-- templates/post_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="post-entry">
  <h2>{{ object.title }}</h2>
  <p>{{ object.body }}</p>
</div>
{% endblock content %}
```

[Classy Class-Based Views](#), a website dedicated to describing the full methods and attributes for each Django GCBV.

CreateView

Code

```
# blog/views.py
from django.views.generic import ListView, DetailView
from django.views.generic.edit import CreateView # new
from .models import Post

class BlogListView(ListView):
    model = Post
    template_name = "home.html"

class BlogDetailView(DetailView):
    model = Post
    template_name = "post_detail.html"

class BlogCreateView(CreateView): # new
    model = Post
    template_name = "post_new.html"
    fields = ["title", "author", "body"]
```

At the top we import **CreateView** from the `django.views.generic.edit` module. Then we create a class, **BlogCreateView**, that extends **CreateView**, specifying the model, **Post**, the template name, **post_new.html**, and the **database fields we want visible on the form**, which are title, author, and body. That's it! The function-based version of this view is much, much longer.

Then, update the URLs file to add a URL route for the create page.

Code

```
# blog/urls.py
from django.urls import path
from .views import BlogListView, BlogDetailView, BlogCreateView # new

urlpatterns = [
    path("post/new/", BlogCreateView.as_view(), name="post_new"), # new
    path("post/<int:pk>/", BlogDetailView.as_view(), name="post_detail"),
    path("", BlogListView.as_view(), name="home"),
]
```

Code

```
<!-- templates/post_new.html -->
{% extends "base.html" %}

{% block content %}
<h1>New post</h1>
<form action="" method="post">{% csrf_token %}
    {{ form }}
    <input type="submit" value="Save">
</form>
{% endblock content %}
```

Let's break down what we've done:

- on the top line, we extended our base template.
- use HTML **<form>** tags with the **POST method** since we're sending data. If we were receiving data from a form, for example, in a search box, we would use GET.
- add a **{% csrf_token %}** which Django provides to protect our form from cross-site request forgery. You should use it for all your Django forms.
- then, **to output our form data use {{ form }}** to render the specified fields.
- finally, specify an **input type of submit** and assign the value "Save".

To view our work, start the server with `python manage.py runserver` and go to the create a new page at `http://127.0.0.1:8000/post/new`.



Django blog

New post

Title:

Author:

Body:

Upon completion, it will redirect to a detail page with the post at `http://127.0.0.1:8000/post/3/`.
Success!



Django blog

3rd post

I wonder if this will work?

Code

```
<!-- templates/base.html -->
{% load static %}
<html>

<head>
  <title>Django blog</title>
  <link href="https://fonts.googleapis.com/css?family=\
    Source+Sans+Pro:400" rel="stylesheet">
  <link href="{% static 'css/base.css' %}" rel="stylesheet">
</head>

<body>
  <div>
    <header>
      <!-- start new HTML... -->
      <div class="nav-left">
        <h1><a href="{% url 'home' %}">Django blog</a></h1>
      </div>
      <div class="nav-right">
        <a href="{% url 'post_new' %}">+ New Blog Post</a>
      </div>
      <!-- end new HTML... -->
    </header>
    {% block content %}
    {% endblock content %}
  </div>
</body>

</html>
```



Django blog

[+ New Blog Post](#)

Hello, World!

My new Blog app.

Goals Today

Learn some Django!

3rd post

I wonder if this will work?

If we were to create this functionality using **function-based views**, we'd need a dedicated **blog/forms.py** file and a **much longer view** explaining how to **handle the form, validate the data, and save it to the database**. By using `CreateView` instead, we rely on **built-in code to handle all these issues**.

```
# blog/views.py
from django.views.generic import ListView, DetailView
from django.views.generic.edit import CreateView, UpdateView # new

from .models import Post


class BlogListView(ListView):
    model = Post
    template_name = "home.html"


class BlogDetailView(DetailView):
    model = Post
    template_name = "post_detail.html"


class BlogCreateView(CreateView):
    model = Post
    template_name = "post_new.html"
    fields = ["title", "author", "body"]


class BlogUpdateView(UpdateView): # new
    model = Post
    template_name = "post_edit.html"
    fields = ["title", "body"]
```

Notice that we are not including “author” in the fields because we assume that the author of a post will not change during editing. We only want the title and body text to be available to update

Code

```
# blog/urls.py
from django.urls import path
from .views import (
    BlogListView,
    BlogDetailView,
    BlogCreateView,
    BlogUpdateView, # new
)

urlpatterns = [
    path("post/new/", BlogCreateView.as_view(), name="post_new"),
    path("post/<int:pk>/", BlogDetailView.as_view(), name="post_detail"),
    path("post/<int:pk>/edit/", BlogUpdateView.as_view(), name="post_edit"), # new
    path("", BlogListView.as_view(), name="home"),
]
```

Code

```
<!-- templates/post_edit.html -->
{% extends "base.html" %}

{% block content %}
<h1>Edit post</h1>
<form action="" method="post">{% csrf_token %}
    {{ form }}
    <input type="submit" value="Update">
</form>
{% endblock content %}
```

At the top, we use `extends` on the base template, `base.html`, and then sandwich this page's content between the content blocks. We again use HTML `<form></form>` tags, Django's `csrf_token` for security, and give it the value "Update" on the submit button.

Code

```
<!-- templates/post_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="post-entry">
  <h2>{{ post.title }}</h2>
  <p>{{ post.body }}</p>
</div>
<!-- start new HTML... -->
<a href="{% url 'post_edit' post.pk %}">+ Edit Blog Post</a>
<!-- end new HTML... -->
{% endblock content %}
```

DeleteView

Code

```
# blog/views.py
from django.views.generic import ListView, DetailView
from django.views.generic.edit import CreateView, UpdateView, DeleteView # new
from django.urls import reverse_lazy # new

from .models import Post

...
class BlogDeleteView(DeleteView): # new
    model = Post
    template_name = "post_delete.html"
    success_url = reverse_lazy("home")
```

DeleteView requires a model, template name, and success URL. We have supplied all three here. Remember, after a blog post is deleted, we have to redirect the user somewhere. In this case, that is our homepage at the URL name of “home”.

You may have noticed that both CreateView and UpdateView also have redirects yet we did not have to specify a success_url. Why is this? If available, Django, by default, will use the `get_absolute_url()` on the model object, a handy feature, but you would only know about it from reading a book like this or deeply reading the docs.

Code

```
# blog/urls.py
from django.urls import path
from .views import (
    BlogListView,
    BlogDetailView,
    BlogCreateView,
    BlogUpdateView,
    BlogDeleteView, # new
)

urlpatterns = [
    path("post/new/", BlogCreateView.as_view(), name="post_new"),
    path("post/<int:pk>/", BlogDetailView.as_view(), name="post_detail"),
    path("post/<int:pk>/edit/", BlogUpdateView.as_view(),
        name="post_edit"),
    path("post/<int:pk>/delete/", BlogDeleteView.as_view(),
        name="post_delete"), # new
    path("", BlogListView.as_view(), name="home"),
]
```

Code

```
<!-- templates/post_delete.html -->
{% extends "base.html" %}

{% block content %}
<h1>Delete post</h1>
<form action="" method="post">{% csrf_token %}
  <p>Are you sure you want to delete "{{ post.title }}"?</p>
  <input type="submit" value="Confirm">
</form>
{% endblock content %}
```

Code

```
<!-- templates/post_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="post-entry">
  <h2>{{ post.title }}</h2>
  <p>{{ post.body }}</p>
</div>
<div>
  <p><a href="{% url 'post_edit' post.pk %}">+ Edit Blog Post</a></p>
  <!-- new HTML below here... -->
  <p><a href="{% url 'post_delete' post.pk %}">+ Delete Blog Post</a></p>
</div>
{% endblock content %}
```



Django blog

[+ New Blog Post](#)

Hello, World! (EDITED)

My new Blog app. Woohoo!

[+ Edit Blog Post](#)

[+ Delete Blog Post](#)



Django blog

[+ New Blog Post](#)

Delete post

Are you sure you want to delete "Hello, World! (EDITED)"?

Confirm



Django blog

[+ New Blog Post](#)

Goals Today

Learn some Django!

3rd post

I wonder if this will work?

Fortunately, Django has a powerful, battle-tested, built-in **user authentication system** that we can use and customize as needed.

Whenever you create a new project, by default, Django installs the **auth** app, which provides us with a **User** object containing:

- username
- password
- email
- first_name
- last_name

We will use this User object to implement login, logout, and signup in our blog application.

Django provides us with a default view for a login page via LoginView. All we need to add is a URL pattern for the auth system, a login template, and a minor update to our `django_project/settings.py` file.

First, update the `django_project/urls.py` file.
We'll place our login and logout pages at the `accounts/` URL: a one-line addition to the next-to-last line.

Code

```
# django_project/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/", include("django.contrib.auth.urls")), # new
    path("", include("blog.urls")),
]
```

As the LoginView documentation notes, by default, Django will look within a templates directory called registration for a file called login.html for a login form. So we need to create a new directory within templates called registration.

Code

```
<!-- templates/registration/login.html -->
{% extends "base.html" %}

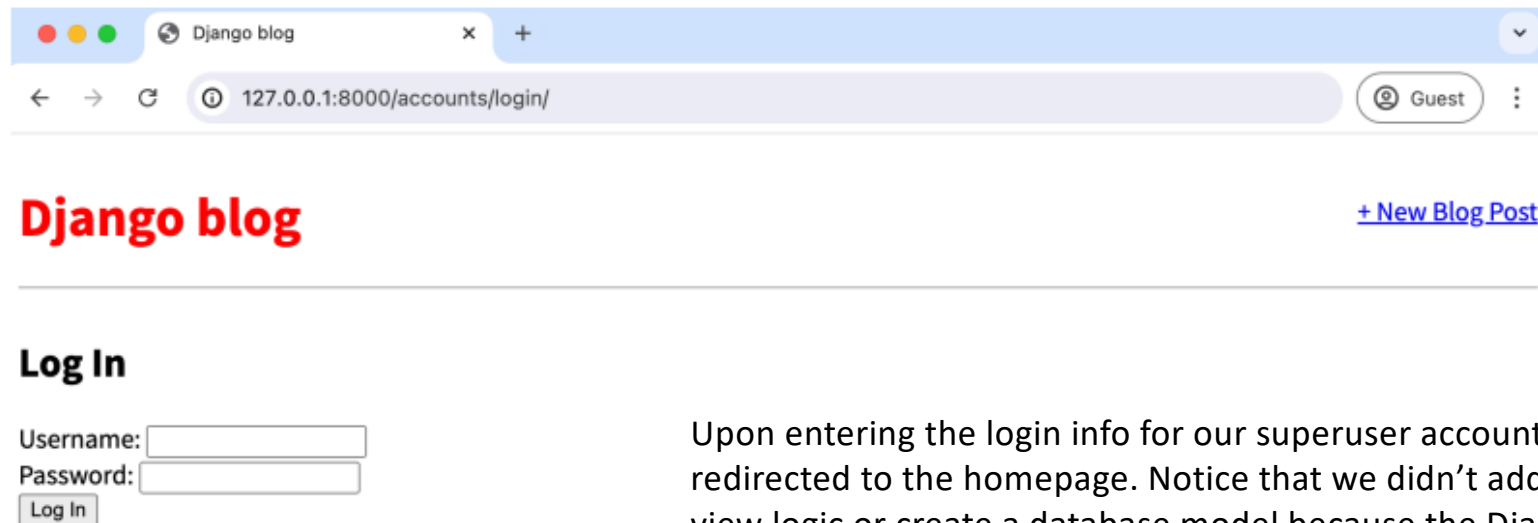
{% block content %}
<h2>Log In</h2>
<form method="post">{% csrf_token %}
    {{ form }}
    <button type="submit">Log In</button>
</form>
{% endblock content %}
```

The final step is to specify *where* to redirect the user upon successful login. We can set this with the `LOGIN_REDIRECT_URL` setting. At the bottom of the `django_project/settings.py` file, add the following:

Code

```
# django_project/settings.py
LOGIN_REDIRECT_URL = "home" # new
```

Now the user will be redirected to the “home” template, which is our homepage. And we’re actually done at this point! If you now start up the Django server again with `python manage.py runserver` and navigate to our login page at `http://127.0.0.1:8000/accounts/login/`, you’ll see the following:



The screenshot shows a web browser window with the title "Django blog". The address bar displays the URL "127.0.0.1:8000/accounts/login/". The page content includes a red header "Django blog" with a link "+ New Blog Post" on the right. Below the header, there is a "Log In" section. This section contains two labels, "Username:" and "Password:", each followed by an input field. A "Log In" button is positioned below the password input field.

Upon entering the login info for our superuser account, we are redirected to the homepage. Notice that we didn’t add any view logic or create a database model because the Django auth system automatically provided both for us. Thanks, Django

Code

```
<!-- templates/base.html -->
...
<body>
  <div>
    <header>
      <div class="nav-left">
        <h1><a href="{% url 'home' %}">Django blog</a></h1>
      </div>
      <div class="nav-right">
        <a href="{% url 'post_new' %}">+ New Blog Post</a>
      </div>
    </header>
    <!-- start new HTML... -->
    {% if user.is_authenticated %}
    <p>Hi {{ user.username }}!</p>
    {% else %}
    <p>You are not logged in.</p>
    <a href="{% url 'login' %}">Log In</a>
    {% endif %}
    <!-- end new HTML... -->
    {% block content %}
    {% endblock content %}
  </div>
</body>

</html>
```



Django blog

[+ New Blog Post](#)

Hi wsv!

Goals Today

Learn some Django!

3rd post

I wonder if this will work?

- Logout
- SignUp